11

SESIÓN 11

IA generativa aplicada a integración: tool calling y arquitecturas de agentes empresariales

Semana 11 · 3 horas (180 min) · Se habilita el Escenario B del Taller 2

11 Objetivos y preparación

OBJETIVOS DE APRENDIZAJE

1. Explicar el mecanismo de tool calling —el modelo decide, el runtime ejecuta— y sus implicaciones de arquitectura
2. Distinguir workflow determinista, workflow aumentado, cadena y agente, y elegir el peldaño mínimo necesario
3. Diseñar el contrato de las herramientas de un agente: granularidad, permisos, idempotencia y presupuesto

LECTURAS PREVIAS (≤ 80 MIN)

- Anthropic, «Building Effective Agents» — workflows frente a agentes
- Huyen, C. AI Engineering — capítulo de agentes
- Opcional: documentación de tool use de un proveedor

ACTIVIDAD DETONANTE (LMS)

La conciliación volumétrica exige restar volúmenes con exactitud. ¿Debe hacer la resta el modelo? Si no, ¿qué hace entonces el agente? Responda en dos líneas.

11

Agenda de la sesión (180 min)

Bloque	Min	Actividad	Objetivo
0	5	Apertura: el dilema de la resta	—
1	40	La mecánica del tool calling y la frontera de ejecución	1
2	40	La escalera de autonomía y el presupuesto del episodio	2
R	**15**	**Receso**	—
3	25	El contrato de la herramienta: anatomía, fallas e idempotencia	3
4	45	Práctica en papel: catálogo de herramientas y trazado de un episodio	3
5	10	Cierre: ¿agente o workflow? y habilitación del Taller 2	—

La práctica se resuelve con lápiz y papel. No hay demostración en vivo: el framework de orquestación llega en la sesión 13.

11 El recorrido de la sesión

De cómo funciona el mecanismo, a cuánta autonomía conceder y con qué contrato

1

La mecánica

El ciclo, la frontera de ejecución y qué se desplaza respecto de la integración clásica

2

La escalera de autonomía

Cuatro peldaños, el criterio para elegir y el presupuesto del episodio

3

El contrato de la herramienta

Anatomía, modos de falla, idempotencia y el prompt de sistema

4

Aplicar

Práctica en papel y habilitación del Escenario B del Taller 2

La tesis de la sesión: no cambia la tecnología de invocación, cambia quién decide invocar y cuándo. Todo lo demás —permisos, pruebas, auditoría— se reacomoda alrededor de ese desplazamiento.

11 El dilema de la resta

Bloque 0 · la pregunta con la que abrimos, y que organiza toda la sesión



POR QUÉ ESTA PREGUNTA IMPORTA

- Un modelo de lenguaje es un sistema probabilístico; una resta de la que depende un reporte regulatorio exige exactitud
- La respuesta no es «el modelo no sirve», sino que el modelo ocupa otro lugar en la arquitectura
- El modelo orquesta, interpreta y redacta; el código calcula, valida y transacciona
- A ese reparto lo llamamos determinismo selectivo, y es la decisión de diseño de la sesión

La pregunta no es qué puede hacer el modelo, sino qué debe hacer el modelo y qué no debe salir del código.

11

El ciclo del tool calling

Bloque 1 · el mecanismo, antes de cualquier consideración de diseño



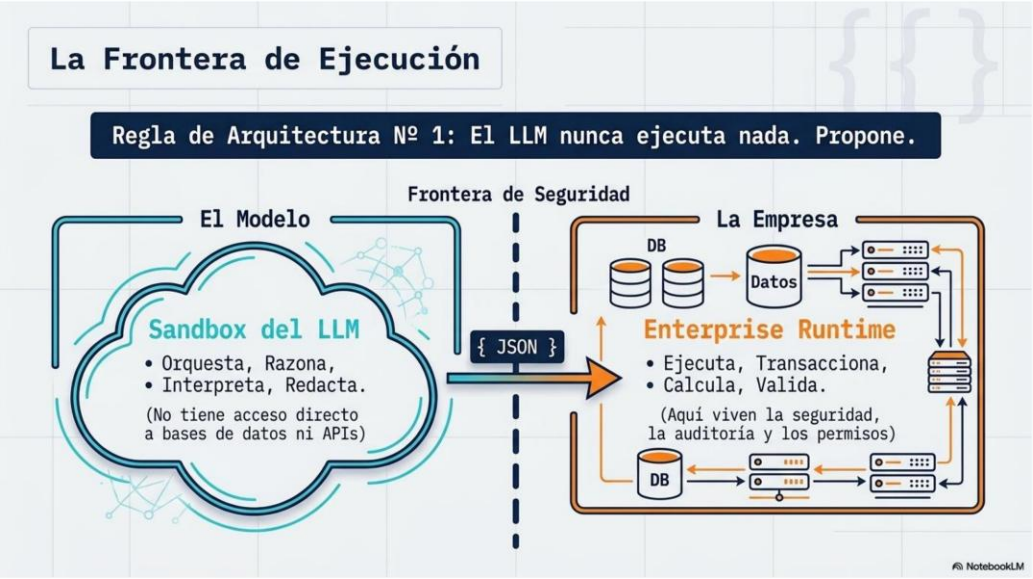
LAS CUATRO ETAPAS

- Contexto: el modelo recibe el objetivo y el catálogo de herramientas con sus esquemas
- Decisión: en lugar de texto, emite una llamada estructurada con nombre y parámetros
- Ejecución: el runtime intercepta esa llamada y ejecuta el código correspondiente
- Actualización: el resultado vuelve al contexto y el ciclo se repite hasta que haya respuesta

El modelo no llama a nada: emite una intención de llamada. Quien llama, siempre, es el runtime.

11 La frontera de ejecución

Bloque 1 · dónde termina el modelo y empieza la empresa



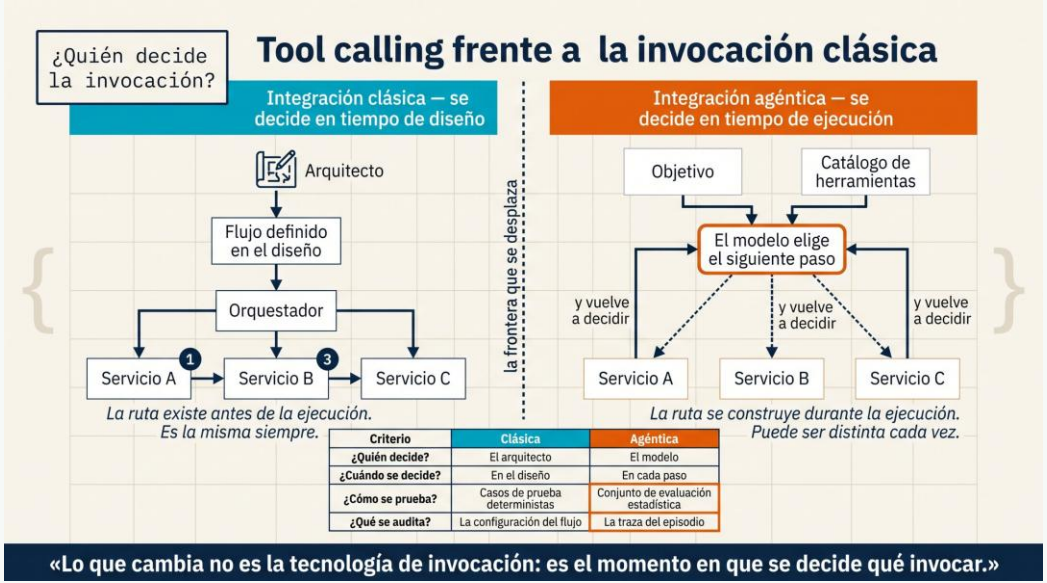
QUÉ VIVE EN CADA LADO

- Del lado del modelo: orquestar, razonar, interpretar y redactar. Ningún acceso directo a datos ni a APIs
- Del lado del runtime: ejecutar, transaccionar, calcular y validar
- Y con el runtime viajan las tres cosas que no se delegan: la seguridad, la auditoría y los permisos
- La frontera se cruza con un mensaje estructurado, que es un punto de control inspeccionable

Regla de arquitectura nº 1: el modelo nunca ejecuta nada. Propone. Por eso el diseño de la frontera es el diseño del sistema.

11 ¿Quién decide la invocación?

Bloque 1 · la lámina que traduce el tema al vocabulario del curso



LO QUE SE DESPLAZA

- En una integración clásica la ruta existe antes de ejecutarse: la trazó el arquitecto y es la misma siempre
- Aquí la ruta se construye durante la ejecución y puede ser distinta en cada episodio
- Cambia cómo se prueba: de casos deterministas a un conjunto de evaluación, como en la sesión 10
- Y cambia qué se audita: ya no la configuración del flujo, sino la traza de cada episodio

La consecuencia incómoda: un flujo que no existe hasta que se ejecuta no se puede revisar en un diagrama. Solo se acota con contratos y presupuesto.

11

Determinismo selectivo: cómo se reparte el trabajo

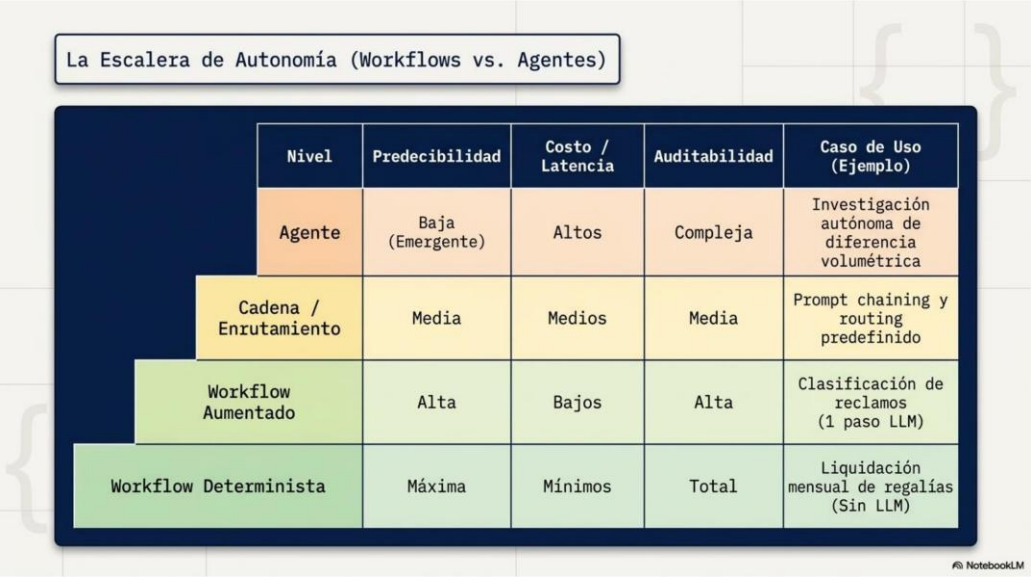
Bloque 1 · la respuesta a la actividad detonante, convertida en criterio de diseño

Qué hay que hacer	Dónde vive	Por qué ahí
Calcular la diferencia volumétrica	Código del runtime	El resultado sustenta un reporte regulatorio: debe ser exacto y reproducible
Validar que el tramo y el período existen	Código del runtime	Es una regla de negocio verificable, no una interpretación
Ejecutar una escritura con efectos	Código del runtime	Exige permisos, registro de auditoría y posibilidad de reversión
Decidir qué consultar a continuación	El modelo	El camino depende de lo que se encuentre en el paso anterior
Interpretar un procedimiento ambiguo	El modelo	Exige juicio sobre texto no estructurado
Redactar el hallazgo con sus fuentes	El modelo	Es producción de lenguaje a partir de material recuperado

La frontera no se traza por dificultad sino por exigencia de exactitud: lo que deba ser exacto, reproducible y auditable no sale del código.

11 La escalera de autonomía

Bloque 2 · cuatro peldaños, y todo lo que crece al subir



QUÉ CRECE EN CADA PELDAÑO

- Sube la capacidad de resolver problemas cuyo camino no se conoce de antemano
- Sube también el costo, la latencia y la dificultad de evaluar el resultado
- Y baja la predecibilidad: el mismo problema puede resolverse por rutas distintas
- La auditabilidad pasa de ser total a ser compleja: hay que reconstruirla desde la traza

No es una escala de madurez. Es una escala de intercambio: cada peldaño compra capacidad con predecibilidad.

11

Los cuatro peldaños, con un caso en cada uno

Bloque 2 · el criterio no es la ambición del proyecto sino la forma del problema

Peldaño	Ejemplo en el caso	Por qué ese y no el siguiente
Workflow determinista	Liquidación mensual de regalías	Las reglas son fijas y el resultado debe ser idéntico cada vez. Sin modelo.
Workflow aumentado	Clasificación de reclamos de mayoristas	Un solo paso necesita juicio sobre texto; el resto del flujo es fijo.
Cadena o enrutamiento	Triaje de una alerta hacia el procedimiento aplicable	Hay varias rutas, todas conocidas de antemano; se elige una.
Agente	Investigación de una diferencia volumétrica anómala	El camino depende de cada hallazgo: no se puede dibujar antes de ejecutarlo.

La prueba práctica: si usted puede dibujar el flujo completo antes de ejecutarlo, no necesita un agente. Y si no puede dibujarlo, tiene que poder evaluarlo — que es la parte cara.

11 El principio que ordena la decisión

Bloque 2 · la frase que queremos que se lleven del curso



CÓMO SE APLICA

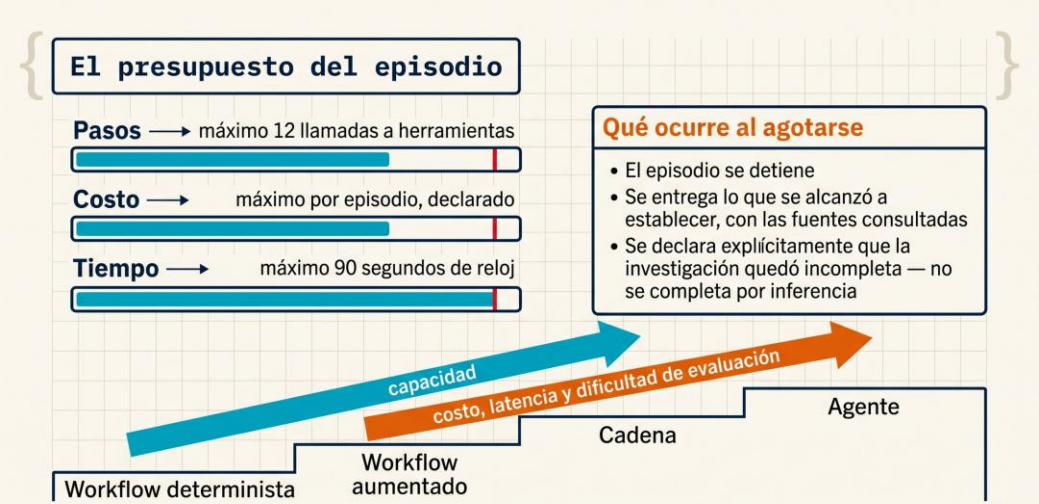
- Empiece por abajo y suba solo cuando el peldaño actual demuestre que no resuelve el problema
- El agente es el último recurso, reservado a los casos donde el camino no se conoce de antemano
- Subir un peldaño sin necesidad no agrega capacidad: agrega costo, latencia y superficie de falla
- Y agrega una obligación nueva: sin evaluación continua, ese peldaño no se puede operar

Use el peldaño más bajo de la escalera de autonomía que resuelva su problema.

11

El presupuesto del episodio

Bloque 2 · lo que convierte la autonomía en algo operable



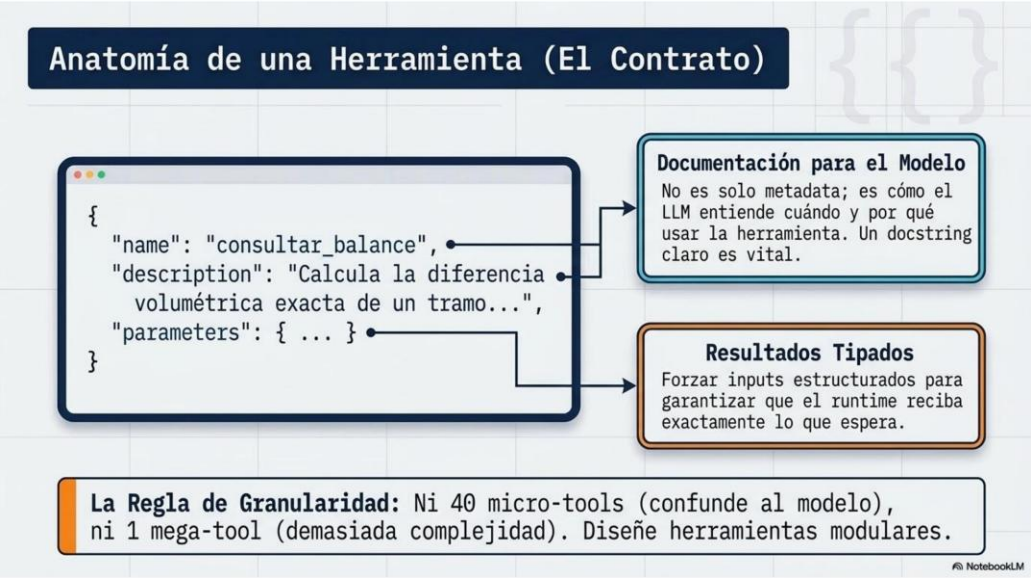
TRES LÍMITES Y UNA CONDUCTA

- Pasos: cuántas llamadas a herramientas puede hacer antes de detenerse
- Costo y tiempo: declarados por episodio, no por consulta individual
- Al agotarse, el episodio se detiene y entrega lo que alcanzó a establecer, con sus fuentes
- Y declara que quedó incompleto: lo que no debe hacer nunca es completar el resto por inferencia

Un agente sin presupuesto declarado no es autónomo: es ilimitado. El presupuesto es parte del contrato, igual que la latencia.

11 Anatomía de una herramienta

Bloque 3 · el contrato que el modelo lee para decidir



LAS TRES PARTES DEL CONTRATO

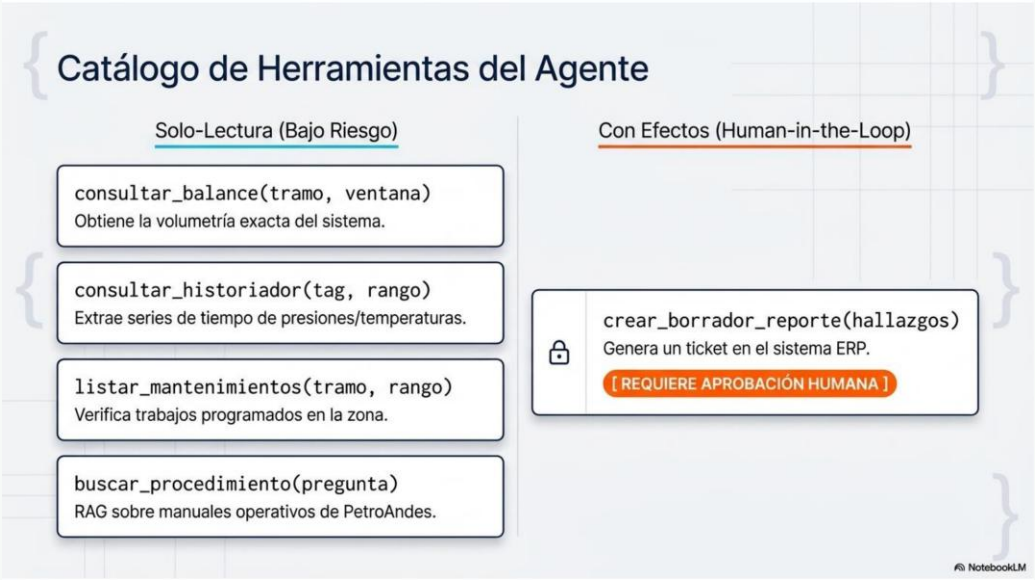
- La descripción no es metadato: es la documentación que el modelo lee para decidir cuándo usarla
- Los parámetros tipados con esquema garantizan que el runtime reciba lo que espera
- El retorno y los errores también son contrato: un error explicativo permite al modelo corregir el rumbo
- Granularidad: ni cuarenta micro-herramientas que confunden, ni una mega-herramienta que lo esconde todo

Una descripción ambigua no es un defecto de redacción: es un defecto de arquitectura que se manifiesta como invocación equivocada.

11

Lectura y escritura no son la misma herramienta

Bloque 3 · la clasificación que decide el modelo de permisos



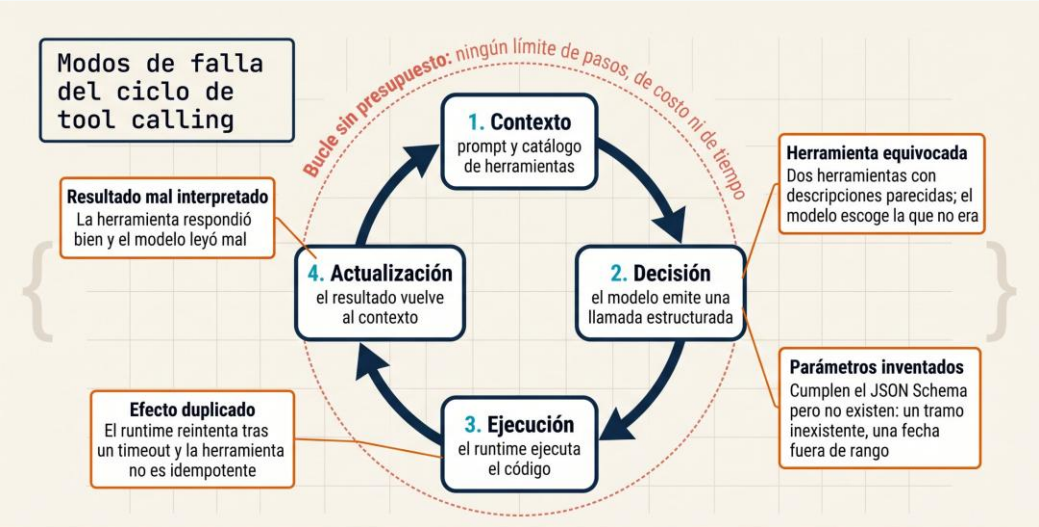
CÓMO SE CLASIFICA

- Solo lectura: bajo riesgo, se ejecuta sin aprobación, pero sigue sujeta a permisos y a registro
- Con efectos: exige aprobación humana explícita antes de ejecutarse
- Cada herramienta declara su nivel, y ese nivel es parte del catálogo, no una decisión del momento
- La matriz herramienta × permiso es un entregable del Taller 2, no un detalle de implementación

El mínimo privilegio se aplica por herramienta, no por agente: el agente no tiene permisos, los tienen sus herramientas.

11 Dónde se rompe el ciclo

Bloque 3 · cuatro modos de falla, y dos de ellos ocurren antes de ejecutar nada



El esquema valida la forma del parámetro, no su verdad. Y el runtime que reintenta no sabe si ya ejecutó.

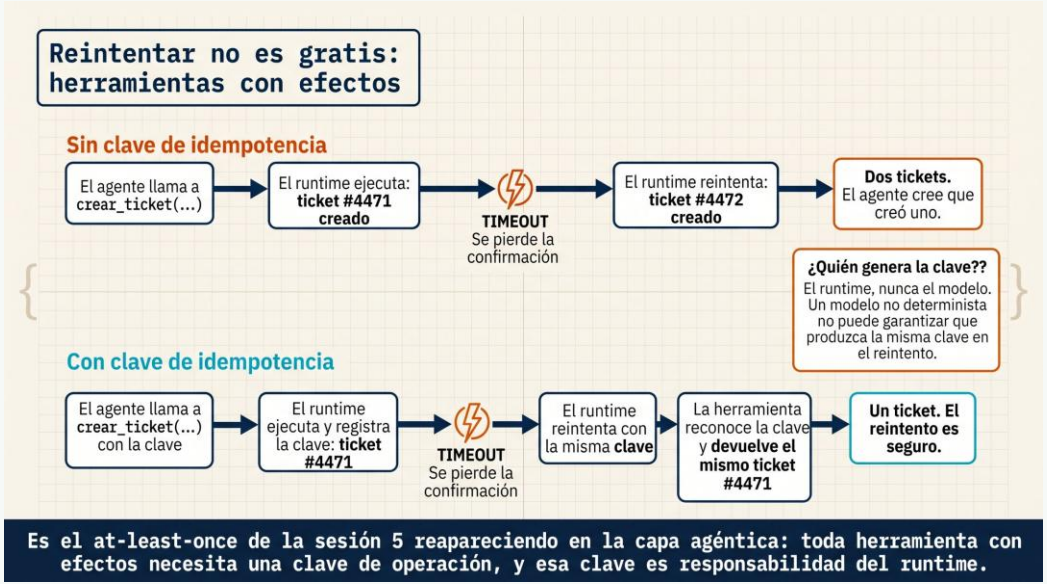
LO QUE HAY QUE RETENER

- Las dos fallas más graves ocurren en la decisión: herramienta equivocada y parámetros inventados
- El esquema valida la forma del parámetro, nunca su verdad: un tramo inexistente puede ser un string válido
- En la ejecución aparece el efecto duplicado, cuando el runtime reintenta algo que no es idempotente
- Y en la actualización, el resultado correcto mal interpretado, que es el más difícil de detectar

Por eso la calidad de la descripción y la validación de dominio en el runtime son decisiones de arquitectura, no de implementación.

11 Reintentar no es gratis

Bloque 3 · el at-least-once de la sesión 5, reapareciendo en la capa agéntica



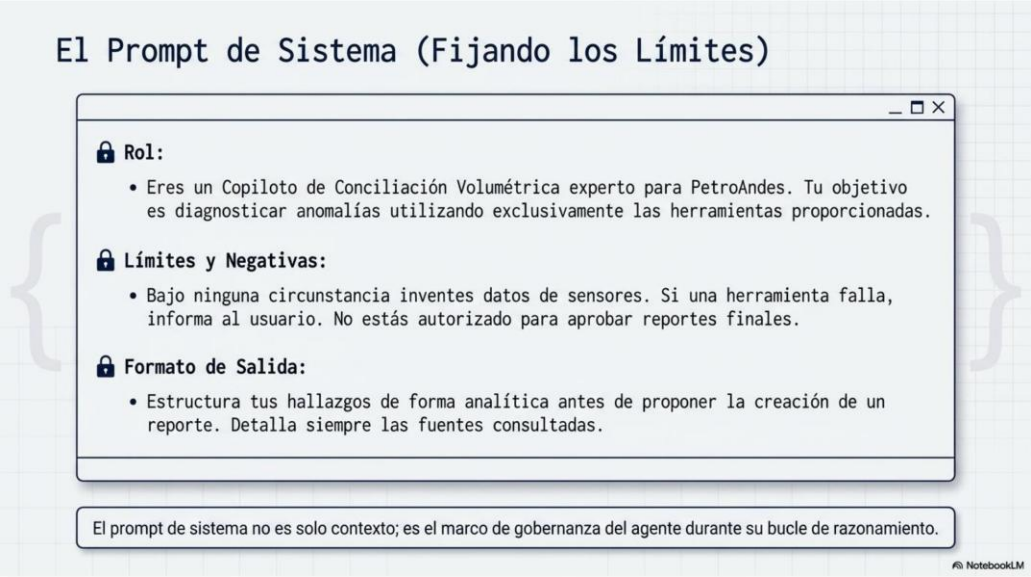
LA DECISIÓN DE DISEÑO

- Si el runtime reintenta tras un timeout y la herramienta no es idempotente, el efecto ocurre dos veces
- La clave de idempotencia permite a la herramienta reconocer el reintento y devolver el mismo resultado
- Quién la genera es la pregunta de arquitectura: el runtime, nunca el modelo
- Un modelo no determinista no puede garantizar que produzca la misma clave en el reintento

Toda herramienta con efectos necesita una clave de operación, y esa clave es responsabilidad del runtime.

11 El prompt de sistema es un marco de gobierno

Bloque 3 · lo que el agente puede, lo que no puede, y cómo debe negarse



QUÉ DECLARA

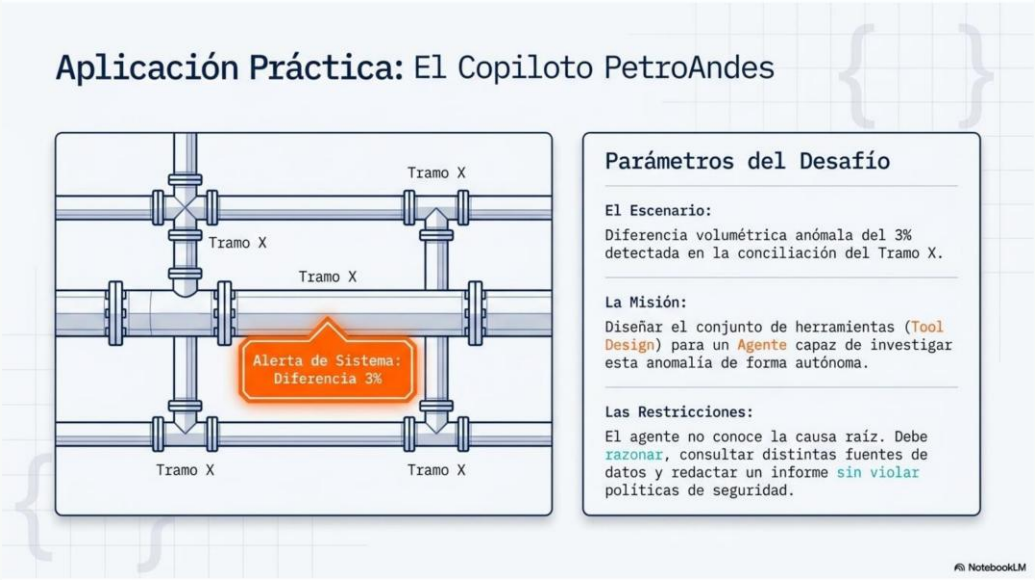
- Rol y alcance: qué problema resuelve y con qué herramientas, exclusivamente
- Límites y negativas: qué no está autorizado a hacer y cuándo debe detenerse
- Formato de salida: estructura declarada, con las fuentes consultadas siempre visibles
- No es contexto adicional: es la parte del contrato que gobierna el bucle de razonamiento

Un límite escrito en el prompt es una declaración de intención; el mismo límite impuesto por el runtime es un control. Hacen falta los dos.

11

El caso: copiloto de conciliación volumétrica

Bloque 4 · el escenario sobre el que vamos a trabajar en papel



LOS PARÁMETROS DEL EJERCICIO

- Se detecta una diferencia volumétrica del 3 % en un tramo, por encima de la tolerancia vigente
- El agente no conoce la causa: debe razonar, consultar distintas fuentes y documentar lo que encuentre
- No accede a sistemas OT: consume datos ya publicados a través de la DMZ, como en el Taller 1
- Y el cálculo de la diferencia no es suyo: viene del sistema de balance, que es el único autorizado

El ejercicio no pide construir el agente, sino escribir el contrato de sus herramientas y trazar un episodio.

11

La forma del entregable: trazar un episodio

Bloque 4 · así se ve una hoja resuelta de la parte B



CÓMO SE LEE ESTA TRAZA

- Cada paso alterna razonamiento del modelo y ejecución del runtime: nunca se confunden
- El razonamiento declara qué pregunta se está respondiendo y por qué esa herramienta
- El runtime devuelve un dato concreto, que es el que condiciona el paso siguiente
- Una traza así es, literalmente, lo que se audita cuando algo sale mal

Si usted no puede escribir la traza en papel, el agente tampoco va a poder recorrerla.

11

Práctica — Lápiz y papel

Bloque 4 · 45 min · en los grupos del Taller 2 · se entrega la hoja al cierre del bloque

PARTE A · EL CATÁLOGO DE HERRAMIENTAS (25 min)

Defina entre cuatro y seis herramientas del copiloto con su contrato completo: nombre, descripción tal como la leerá el modelo, parámetros, retorno y errores. Clasifique cada una como de lectura o con efectos, y para las que tengan efectos declare el nivel de aprobación, la clave de idempotencia y quién la genera.

PARTE B · TRAZAR EL EPISODIO (20 min)

Para la diferencia del 3 %, escriba la secuencia de llamadas que haría un buen agente, con el razonamiento entre cada una. Marque el paso exacto donde se detiene a pedir aprobación humana, y qué debe ver el aprobador para decidir en menos de dos minutos.

En la puesta en común: «enviar el reporte al regulador» no es una herramienta con aprobación. Distinguir aprobar de ejecutar es el objetivo del ejercicio.

11

Cierre: ¿por qué esto es un agente y no un workflow?

Bloque 5 · el criterio del Parcial 2 y lo que se habilita hoy del Taller 2

EL CRITERIO, EN UNA LÍNEA

Si el camino se conoce de antemano, es un workflow. Si el sistema debe decidir qué consultar a partir de hallazgos intermedios, es un agente.

- La mecánica: el modelo orquesta, el runtime ejecuta transacciones exactas
- La autonomía: usar siempre el peldaño más bajo que resuelva el problema
- El contrato: descripciones precisas, permisos por herramienta e idempotencia declarada

QUÉ SE HABILITA HOY

Del Taller 2, el Escenario B: el catálogo de herramientas, la matriz herramienta × permiso y la frontera determinista.

Es exactamente lo que acaban de hacer en papel. El resto del Escenario B se habilita en las sesiones 13 a 16.

La sesión 12 responde la pregunta que queda abierta: si cada agente define sus propias herramientas, volvemos al problema de integración N × M que ya resolvimos una vez.

La mecánica | La escalera de autonomía | El contrato de la herramienta | **Aplicar**

11

Bibliografía de la sesión

LECTURAS OBLIGATORIAS

- Anthropic. Building Effective Agents, 2024 — la distinción entre workflows y agentes, y los patrones de composición. Es el ensayo canónico del tema.
- Huyen, C. AI Engineering: Building Applications with Foundation Models. O'Reilly, 2025 — capítulo de agentes y diseño de herramientas.
- Hohpe, G. y Woolf, B. Enterprise Integration Patterns, 2003 — Routing Slip y Process Manager: el vocabulario clásico del enrutamiento dinámico, que es lo que el bucle agéntico reinventa.

OPCIONALES

- Documentación de tool use o function calling de un proveedor — el formato concreto de la llamada estructurada.
- Nygard, M. Release It!, 2018 — timeouts, reintentos e idempotencia: el sustento de la lámina del reintento.
- Especificación de MCP, modelcontextprotocol.io — puente directo a la sesión 12.
- Yao, S. et al. ReAct: Synergizing Reasoning and Acting in Language Models. arXiv:2210.03629, 2022 — el origen del bucle razonar-actuar.